

Week 4: Inference – Variable Elimination and Belief Propagation

Practical Assignment

For this week’s practical, we revisit two models from Week 2 that can be represented as factor trees: the student network and the casino HMM. We will implement the following inference algorithms:

- Variable elimination (VE)
- Belief propagation (BP)

For both VE and BP, we will implement the algorithms manually; that is, we will calculate each step in the algorithms using factor operations. We will also use `emdw`’s built-in functionality to do BP for us.

1 The student network revisited

This question revisits the student network that we used in Question 2 of Week 2’s practical. In this question, you should use and build on the work you did for Question 2.1(a) of Week 2’s practical.

1.1 Variable elimination (VE)

The goal of this question is to implement the VE algorithm for the student network using `emdw`’s factor operations.

- (On paper) Draw the FG for the student network with no RVs observed.
- (On paper) We want to calculate the same marginal posterior belief distribution as in Question 2.1(b) of Week 2; that is, $p(L|D = 1, S = 1)$. Now (symbolically) insert the observed evidence, $D = 1$ and $S = 1$, into the factors of the model¹, and draw the resulting FG – you should get something that looks like Figure 1.

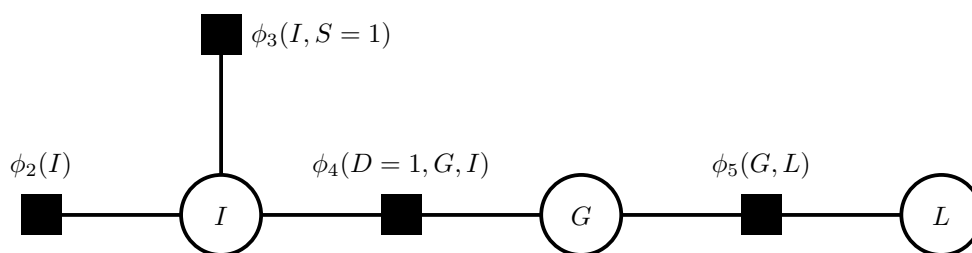


Figure 1: FG for the student network where $D = 1$ and $S = 1$ have been observed. The factors are defined as: $\phi_1(D) = p(D)$, $\phi_2(I) = p(I)$, $\phi_3(I, S) = p(S|I)$, $\phi_4(D, G, I) = p(G|D, I)$ and $\phi_5(G, L) = p(L|G)$.

- (On paper) To calculate the desired marginal belief, $p(L|D = 1, S = 1)$, using VE, we have to choose a variable elimination ordering. The obvious elimination ordering is first I and then G . Now describe symbolically what calculations are performed for the elimination of each RV. For example, the marginalisation

¹Note that the idea in this question is not to calculate the tables by hand (although you might choose to do so for practice), but rather just to name the factors and specify which RVs they contain; i.e., just write down the factors symbolically, similarly to how they are written in Figure 1.

of I can be “pushed into” the product of factors as follows:

$$\begin{aligned}
 p(L|D = 1, S = 1) &\propto \sum_G \sum_I \phi_2(I) \phi_3(I, S = 1) \phi_4(D = 1, G, I) \phi_5(G, L) \\
 &= \sum_G \phi_5(G, L) \left[\sum_I \phi_2(I) \phi_3(I, S = 1) \phi_4(D = 1, G, I) \right] \\
 &= \sum_G \phi_5(G, L) \phi_6(G).
 \end{aligned} \tag{1}$$

This means that during the elimination of I , a new factor

$$\phi_6(G) = \sum_I \phi_2(I) \phi_3(I, S = 1) \phi_4(D = 1, G, I) \tag{2}$$

replaces the factors $\phi_2(I)$, $\phi_3(I, S = 1)$ and $\phi_4(D = 1, G, I)$. Also draw the FG after each RV has been eliminated.

- (d) (In code) Implement the VE procedure of point (c) using factor operations in `emdw` (remember to normalise your final result). Compare your answer to that of Week 2’s Question 2.1(b).
- (e) (On paper/in code) Determine the size of the largest table (i.t.o. the number of table entries) that you created during the VE procedure in point (d). Compare this with the size of the joint distribution you created in Week 2’s Question 2.1(a). Can you see the advantage of VE?
- (f) (On paper/in code) Repeat points (c) and (d), but now calculate the marginal belief $p(I|D = 1, S = 1)$. Can you see the disadvantage of VE when you want to extract multiple marginal beliefs given the same evidence?

1.2 Manual belief propagation (BP)

The goal of this question is to implement the BP algorithm using `emdw`’s factor operations.

- (a) (On paper) Redraw the FG of Question 1.1(b) (i.e., the FG that corresponds to the factors of the model where the evidence, $D = 1$ and $S = 1$, has been incorporated), but also indicate all the messages required for BP in a FG. Your graph should look something like Figure 2.

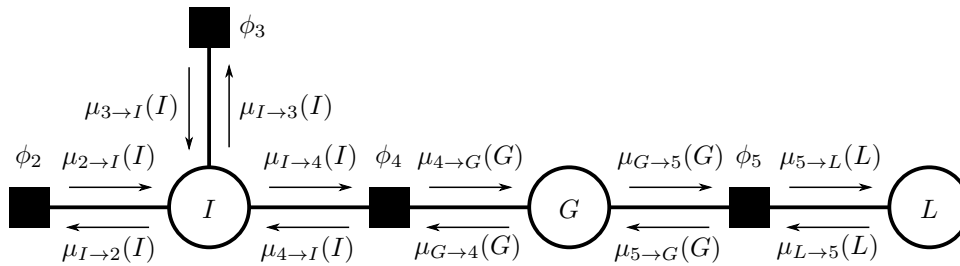


Figure 2: FG for the student network where $D = 1$ and $S = 1$ have been observed, with all the messages required for BP in a FG indicated.

- (b) (On paper) Determine an appropriate message-passing order. Remember that before an outgoing message from a node can be calculated, the node must have received all the messages necessary to calculate the outgoing message.
- (c) (In code) Create and calculate all the messages in the FG, using factor operations in `emdw`, in the order you determined in point (b).
- (d) (In code) Calculate the marginal posterior belief $p(L|D = 1, S = 1)$ by gathering the incoming messages to the *variable node* L . Check that it is the same as for Question 1.1(d).
- (e) (In code) Calculate the marginal posterior belief $p(L|D = 1, S = 1)$ by gathering the incoming messages to the *factor node* 5, multiplying them with the factor $\phi_5(G, L)$, and marginalising out G . Check that it is the same as for point (d).

- (f) (In code/on paper) For each of the messages sent towards the variable node L (i.e., $\mu_{2 \rightarrow I}(I)$, $\mu_{3 \rightarrow I}(I)$, $\mu_{I \rightarrow 4}(I)$, $\mu_{4 \rightarrow G}(G)$, $\mu_{G \rightarrow 5}(G)$ and $\mu_{5 \rightarrow L}(L)$ in Figure 2), determine the equivalent quantities in the symbolic VE process of Question 1.1(c). Also numerically check some of these messages against the tables produced during the VE process that was implemented in code in Question 1.1(d).
- (g) (In code/on paper) Determine the size of the largest table created during the BP inference procedure. Compare this with that of VE you determined in Question 1.1(e). Does BP have any advantage over VE in terms of computational complexity?
- (h) (In code) Calculate the marginal belief $p(I|D=1, S=1)$ from the messages in the FG. Compare the result with that of Question 1.1(f). Can you see the advantage of BP over VE when you want to extract multiple marginal beliefs given the same evidence?

1.3 Belief propagation (BP) using emdw's built-in functionality

You now have experience in what is involved in BP message passing. Let's now automate things by letting emdw do the work.

Some declarations you will need: In addition to the headers already included in `example.cc`, add the following headers close to the top of your *.cc file:

```
// emdw headers
#include "clustergraph.hpp"
#include "lbp_cg.hpp"
#include "messagequeue.hpp"
```

Passing the model and the evidence to emdw: emdw can do all the work for you to insert the evidence into the factors of the model, and construct a graph used for inference. To achieve this, you need to do the following:

- Create a vector container to hold all the Factors of the model:

```
vector< rcptr<Factor> > factorPtrs;
```

Typically we use the `push_back` function to add the `rcptr<Factor>`s to the back of this vector. For example, if you have created a pointer to a factor called `ptrI`, then you can add this factor to the vector `factorPtrs` by writing

```
factorPtrs.push_back(ptrI);
```

You have to add all the factors of the model to this vector. Do not incorporate the evidence into these factors yet – emdw will do that for you (see below). The order in which you add the factors is unimportant.

- You also need to pass all the observed RV values to emdw. To do this, create a map (or dictionary) as follows:

```
map<RVIdType, AnyType> obsv;
```

The key/first part (`RVIdType`) identifies the particular RV, while the value/second part (`AnyType`) specifies its observed value. You should now add all the observed RV values to this map. For example, if RV `D` has an observed value of `1`, you can write:

```
obsv[D] = int(1); //or T(1). Note explicit type
```

- We now have to pass the factors and evidence to emdw; we do this as follows using the `ClusterGraph` class²:

```
ClusterGraph cg(ClusterGraph::BETHE, factorPtrs, obsv);
```

Perform inference: emdw can now automatically perform inference.

- First initialise the messages and a structure called a *message queue* (we will encounter this later in the module):

²We will encounter cluster graphs next week; for now, you can assume that emdw builds a FG if you give it the `BETHE` flag (a *Bethe cluster graph* is the same thing as a FG).

```
map<Idx2, rcptr<Factor> > msgs;
MessageQueue msgQ;
```

- Perform BP message passing:

```
unsigned nMsgs = loopyBP_CG(cg, msgs, msgQ);
cout << "Sent " << nMsgs << " messages\n";
```

Query the graph:

- You can now extract the marginal posterior belief for a particular RV. (You can also extract the joint posterior belief of several RVs by specifying more than one variable, separated by commas. However, you can only do this for RVs that occur together in some factor.) For example, if you want to extract the marginal posterior belief over RV L , you can write:

```
rcptr<Factor> qPtr = queryLBP_CG(cg, msgs, {L})->normalize();
```

The pointer `qPtr` will now point to a factor that represents this posterior belief.

- (In code) Use `emd`'s built-in functionality to do BP message passing for the student network with the evidence $D = 1$ and $S = 1$.
- (In code) Extract the posterior belief distribution $p(L|D = 1, S = 1)$. Easy, isn't it? Compare this with your answer for Question 1.2(d).
- (In code) Extract the posterior belief distribution $p(I|D = 1, S = 1)$ and compare it with your answer for Question 1.2(h).

2 The casino HMM revisited

This question revisits the casino HMM that we used in Question 3 of Week 2's practical. You can again reuse and build on the work you did for Question 3.1(a) and (b) of Week 2's practical.

After doing Question 1, you should understand the VE algorithm as well as the fact that VE has no real advantage over BP. So for this question we will skip VE and only perform BP.

2.1 Manual belief propagation (BP)

For this first part of Question 2, we will use the same setup as in Question 3 of Week 2's practical: the casino HMM with 3 time steps. Assume that the RVs $y_1 = Y_1$, $y_2 = Y_2$ and $y_3 = Y_3$ are observed.

- (On paper) Symbolically insert the evidence into the factors of the model, and draw the resultant FG. Your FG should look something like the one in Figure 3.

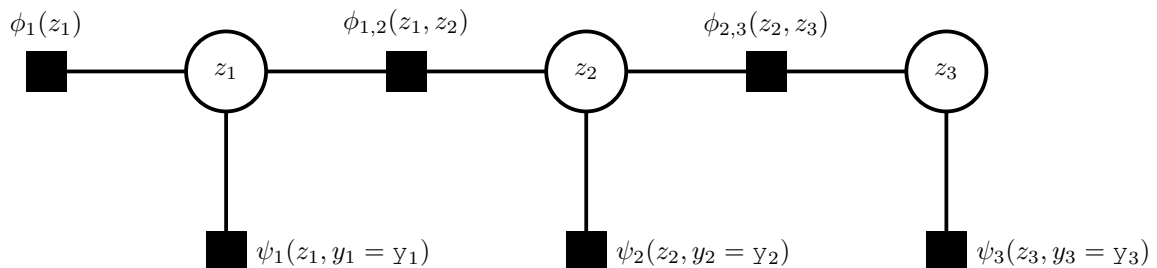


Figure 3: FG for the casino HMM where $y_t = Y_t$ has been observed, for $i = 1, 2, \dots, T$. The factors are defined as: $\phi_1(z_1) = p(z_1)$, $\phi_{t-1,t}(z_{t-1}, z_t) = p(z_t|z_{t-1})$, and $\psi_t(z_t, y_t) = p(y_t|z_t)$.

- (b) (On paper) Draw and label all the messages required for BP in the FG of point (a) (there should be 16 of them). Now determine an appropriate message-passing order. Remember that before an outgoing message from a node can be calculated, the node must have received all the messages necessary to calculate the outgoing message. Also try to find an order that follows a regular pattern. For example, for the variable node z_1 , first send the incoming message from the left (from ϕ_1), then send the incoming message from the bottom (from ψ_1), then send the outgoing message to the right (to $\phi_{1,2}$); you can repeat this pattern for variable node z_2 and variable node z_3 ³. If you follow such a pattern, then it would be possible to implement message passing as a loop iterating over nodes of the graph⁴.
- (c) (In code) Construct all the factors for this model, and observe and reduce the factors that contain RVs y_1, y_2 or y_3 with the evidence $(y_1, y_2, y_3) = (6, 5, 6)$.
- (d) (In code) Calculate all the messages in the FG, using factor operations, in the order you determined in point (b).
- (e) (In code) Calculate the marginal posterior beliefs $p(z_1|y_1 = 6, y_2 = 5, y_3 = 6)$, $p(z_2|y_1 = 6, y_2 = 5, y_3 = 6)$ and $p(z_3|y_1 = 6, y_2 = 5, y_3 = 6)$ from the factors of the model and the messages that you calculated in point (d). Check that these beliefs are the same as the ones that you calculated in Question 3.1(d) of Week 2's practical.
- (f) (In code/on paper) Determine the size of the largest table that you created during the BP inference procedure. Compare this with the size of the joint distribution you created in Question 3.1 of Week 2's practical. Can you see the advantage of BP for the HMM model?
- (g) (On paper) How would the amount of *storage* and *computation* required for BP increase as the number of time steps in the HMM increases?
- (h) (On paper) Can you identify any messages that are unnecessary to send/calculate (if the goal is to calculate the marginal posterior beliefs over every z_i)? And if goal is to only calculate the marginal posterior belief over the latent RV of the *last* time step, z_T , which messages are unnecessary to send/calculate?

2.2 Belief propagation (BP) using emdw's built-in functionality

For this second part of Question 2, we will not use only 3 time steps, but rather use a large number of time steps. Although it is not too difficult to perform manual BP for many time steps (it is pretty much just the implementation of the forward-backward algorithm), we will use emdw's built-in functionality to do the inference for us.

- (a) (On paper) Assume that the number of time steps in the HMM is given by the (for now unspecified) number T . In preparation for the implementation in code for a large number of time steps, decide on a scheme for specifying the RV identities; that is, the unique integer values that the code uses to refer to the RVs in the model. It will obviously not work to assign RV identities with our previous approach of using `enum`. You *could* use the map (dictionary) approach of the image denoising problem of Question 2 in Week 3's practical, but it is unnecessarily cumbersome for the HMM problem. The easiest approach is just to allocate RV identities using something like the following scheme:

- The identity of z_t is $2t - 2$ for $t = 1, \dots, T$.
- The identity of y_t is $2t - 1$ for $t = 1, \dots, T$.

This will assign a unique identity to all $2T$ RVs in the model, and will allow you to easily refer to the correct RV identity for any given time step t .

- (b) (In code) For this question, we will read the observed values for RVs $y_1, \dots, y_T$ from a text file. There are two such text files: `casino_data1.txt`, which contains data for 3 time steps and is the same as that of Question 2.1, and `casino_data2.txt`, which contains data for 300 time steps⁵. Write the code to read

³Can you see that this is similar to the forward pass of the forward-backward algorithm for HMMs? And can you see that the remaining messages would be similar to the backward pass?

⁴We will not use a loop to iterate over the nodes of the graph though – for only 3 time steps, it will be much easier just to manually calculate each message. emdw has the built-in functionality to iterate over the nodes of the graph and send messages – we will use this built-in functionality in Question 2.2 when we increase the number of time steps.

⁵I suggest you use the first (small) data file when you develop your solution – since there are only 3 time steps, it is easy to check things at intermediate steps, and you can validate your results by comparing with that of Question 2.1. If you have working code in place, then you can switch over to the second (larger) data file.

the data from the text file into a `vector` container. The easiest way to do this is to open the text file as an input file stream, e.g.:

```
std::ifstream dataFile("casino_data1.txt");
```

To be safe, you should check that the file has been opened⁶. After this, you can read the contents of the file into a `vector` container `data` using iterators as follows:

```
std::vector<int> data(std::istream_iterator<int>(dataFile),
                    std::istream_iterator<int>{});
```

After populating the `vector` elements, you can get the size of the `vector` by writing `data.size()`, and access any element of the `vector` by writing `data[i]`, where `i` is in the range 0 to `data.size()-1`.

- (c) (In code) Similarly to Question 1.3, create a `vector` container called `factorPtrs` to hold all the factors. For the given number of time steps T (that you can get from the size of the data `vector` in point (b)), create all the factors of the model, using your RV identity scheme of point (a). As you create each factor, add it to `factorPtrs`.
- (d) (In code) Similarly to Question 1.3, create a `map` container called `obsv`. Then add the observations stored in `data` one-by-one to this `map`, using your RV identity scheme of point (a) for the keys.
- (e) Now perform BP using `emdw`'s built-in functionality, similarly to Question 1.3.
- (f) For each time step, extract the marginal posterior belief that the casino is using a loaded die; i.e., $p(z_t = 2|\text{evidence})$. Write these marginal posterior beliefs to a text file (see Question 2 of Week 3's practical for example code). Plot the marginal posterior beliefs (e.g. using Python)⁷. The ground truth (whether the casino used a loaded die at each time step) for the two datasets is available in `casino_ground_truth1.txt` and `casino_ground_truth2.txt` – you can plot the posterior beliefs and ground truth together – the results for the `casino_data2.txt` dataset should be similar to what is shown in Figure 4. Do these results make sense to you?

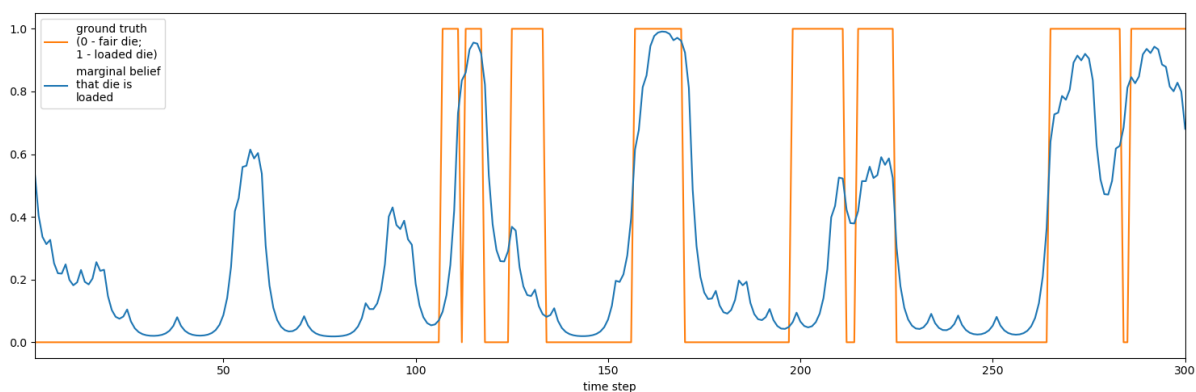


Figure 4: The results for the `casino_data2.txt` dataset.

3 Assessment: Class test

There will be short, written class test at the end of the practical period in the practical venue (see STEMLearn for details). If you have worked through the practical assignment and understood everything, then the test should be easy!

⁶As for the image denoising question of Week 3's practical, make sure that the text file is located in the same folder as the *compiled application* for this question (typically `/devel/emdw/build/src/bin/`). Alternatively, you can specify the absolute path of the text file.

⁷I can suggest the `numpy` function `loadtxt` to load data from a text file into a `numpy array`.